

P04: Makers Makin' It, Act II – The Sequel

Roster: Sarah Zou (PM), Joyce Lin (Dev1), Wesley Leon (Dev 2), Mottaqi Abedin (Dev 3)

Description:

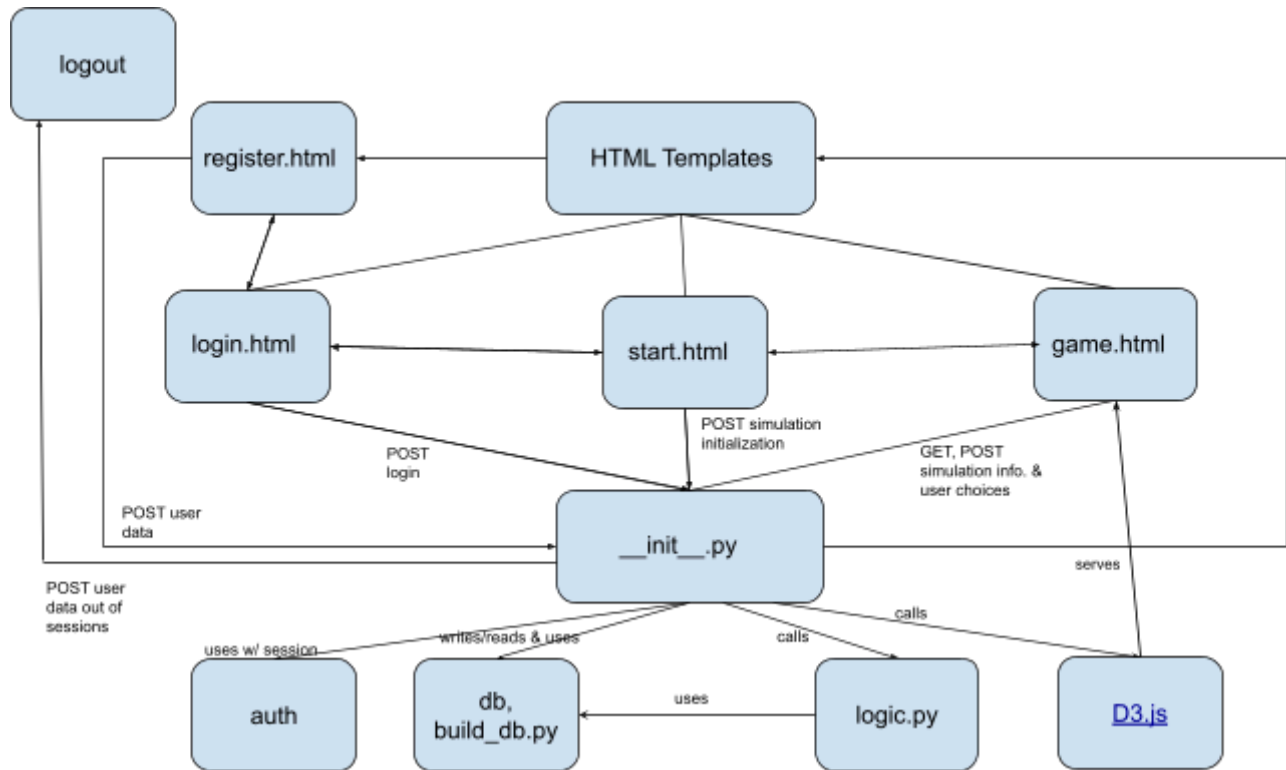
This project is an interactive, story-like Pandemic Simulator that incorporates both user data exploration and visualization to understand how different non-health factors and actions like population density, policy, and environmental and social vulnerability affect outbreaks Using Covid-19 data, the simulator has base outbreak infection and mortality rates that change based on user decisions and actions and are weighted based on correlations between the outbreak data and the factors, as well as some level of randomness.

Components:

- Webpage (frontend)
 - Tailwind for framework
 - Jinja2 HTML templates and user interface: login/registration, landing/start page, simulation page, logout, result and profile page
 - login/registration page automatically loaded if user not in session, otherwise redirect to landing, logout handles session removal, POST user authentication and session details to Flask app
 - landing/start page implements user interface, includes a short introduction, start button, and an US SVG map where user can select and view state statistics, POST user selection and simulation initialization to Flask app
 - simulation page implements user interface, top bar for total stats, date/ticks, side bar for user decisions, container for maps/charts visualizing changing data with [D3.js](#), GET and POST simulation information and updates with user input from and to Flask app
- User Authentication (backend)
 - SQLite users table stores user_id PK and password pairs (SQLite instead of mongodb because most of our data is relational and standardized, not requiring a document)
 - Flask session and [build_db.py](#) module w/ functions to create, check, and match user and password information
- Database (backend):
 - datasets: COVID-19 number infected and deaths by location over time, Social Vulnerability Index and population/population density by location,

Stringency Index (tracks and rates countries' policy response to Covid-19) by location over time

- scale: country, by state
- 5 tables: users (user_id TEXT PK), country (country_id TEXT PK), stringency (stringency_id INT PK, country_id TEXT FK), covid_stats (stats_id INT PK, country_id TEXT FK)
- build_db.py module also initializes db, data cleaning, processing, and query/insert/remove helper functions
- We have our dataset CSVs cleaned and parsed into SQL tables in our build_db.py file as well as getter functions. For each tick, state, covid, social vulnerability, and stringency (policy rating) data are all first fed into logic.py, which imports build_db.py, where we run our calculations on the data for each tick (one week in the simulation) to get the infection/death rates. For every 0.01% of the state's population that is infected, we create a new point object (we use d3.geoBounds on the state feature and generate a coordinate for it, which we use a projection to convert from lat and long coordinates into SVG coordinates) and those new point objects are pushed into a fixed array, exported from our JavaScript for the map. We pass the new array with the new point objects from Flask to game.html to map.js, where we use D3 to update the map and draw the next point by iterating through the array until the new points.
- Additional Helper Modules (backend):
 - logic.py for data and rate calculation (normalizing, averages, etc,) functions and for maintaining and updating simulation states, stats, and narration using build_db.py
 - .js files for D3.js for map and graphs visualizing data over time
- Flask App (middleware):
 - handles routing, HTML templates, calls logic.py and .js files

Component Map:**Database Organization:**

users		
TEXT	user_id	PK
TEXT	password	

states		
TEXT	country_id	PK
INTEGER	vulnerability_index	
INTEGER	population	
FLOAT	population_density	

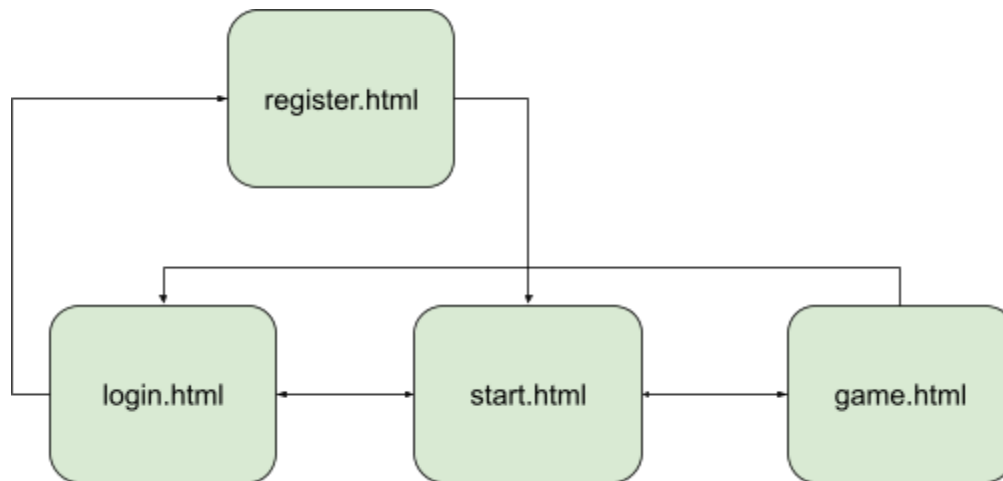
stringency		
------------	--	--

INTEGER	stringency_id	PK
TEXT	country_id	FK
DATE	date	
FLOAT	stringency_index	

covid_stats		
INTEGER	stats_id	PK
TEXT	country_id	FK
DATE	date	
INTEGER	infected	
INTEGER	dead	

spread_points		
INTEGER	point_id	PK
INTEGER	run_id	FK
TEXT	state	
DATE	date	
INTEGER	total_dead	

runs		
INTEGER	run_id	PK
TEXT	user_id	FK
TEXT	country_id	FK
INTEGER	total_infected	
INTEGER	total_dead	

Site Map:**Task Breakdown:**

Sarah (PM):

- coordinate team check-ins, updates, and help/debug where needed
- build_db.py
- [D3.js](#) with joyce
- [logic.py](#) with Wesley

Joyce:

- Flask
- [D3.js](#)

Wesley:

- auth
- [logic.py](#)

Mottaqi:

- html & css/framework
- handled SVGs

FEF: Tailwind

- Tailwind is our chosen FEF for P04 as it allows for high degrees of customizability as opposed to its competitors such as Bootstrap and Foundation. While its selection of usable component samples is decidedly limited (with a majority being unfortunately locked behind a paywall), we will not likely require anything complicated in regards to CSS in this project.
- Specific tools we pulled from this framework include its flex-grid, color and size classes, as well as possible animations (likely purely for aesthetics as animations involved with data visualization will be handled with D3.js)

APIs:

- None!

Data Visualization Library: [D3.js](#)

- We chose to use D3.js to generate our graphs. Though its sample code is a lot harder to navigate than those offered by Chart.js (and also a lot longer), it allows for the visualization of data in much more complex ways including maps (world, stars, etc.), trees, color gradations, and the like. For our project particularly which has changing data and rates, the dynamic visualization of [D3.js](#) is a significant reason why we chose it. On the other hand, Chart.js has a much more simplistic appearance which we believe will be unable to deliver the effect we are looking for.